

Notes 7: Advanced Regression

Table of contents

Introduction	108
Quick Regression Overview/Review	108
Linear Regression Assumptions and Diagnostics	111
Regression Remedial Measures	113
Transform	113
Bootstrapping	113
Weighted Least Squares	116
Regularization – Ridge, LASSO, and Elastic Net	122
Bias-Variance Trade Off	122
Ridge Regression	123
LASSO Regression	124
LASSO vs Ridge Regression	124
Elastic Net Regularization	124
Regression Regularization in R: <code>glmnet</code>	125
Fitting Model with Selected λ	128
LASSO vs Stepwise Selection	128
Ridge Regression Example	129
<code>glmnet</code> in the <code>caret</code> package	132
Example: Logistic Regression Elastic Net	133
Poisson Regression	135
Poisson Link Function	136
Poisson Regression Assumptions	137
Finding Parameter Estimates	137
Fitting the Poisson Model in R	139
Illustrative Example - Lumber	139
Interpreting the Model Coefficients	141
Confidence Intervals for the Model Coefficients	142
Prediction Intervals	143
Poisson Residuals	144
Poisson Regression Diagnostics	144
Quasi-Poisson Regression	147
Quasi-Poisson Example - Quasi-Poisson Output	148

Introduction

In MATH 2140 or 3150 - Applied Statistics, we discussed many topics in regression. We'll discuss a few more advanced regression techniques:

- Weighted least squares
- Regularization
 - Ridge regression
 - LASSO
 - Elastic net
- Poisson regression
- Negative binomial regression

Quick Regression Overview/Review

A linear regression model is one in which we fit a line (in simple linear regression) or a hyperplane (in multiple linear regression) with two primary goals in mind:

- To get predictions of the response variable, Y , using the explanatory variable(s), X (sometimes $X_1, X_2, \dots, X_p$).
- To quantify the nature and the magnitude of the association between Y and the predictor(s). That is, understand what the slope values are telling us.

In general, the regression model can be written

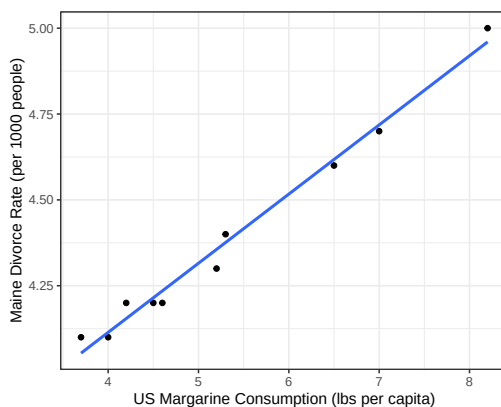
$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \varepsilon_i$$

We can have p predictor variables, each of which has its own slope parameter: the β_j in front of it. In addition, there is an intercept β_0 . So, all together, there are $p + 1$ parameters in this model. The ε is an error term. If we had data for the entire population, then we could compute these β parameters exactly, but we usually do not. That means we have to estimate them. The estimated parameters are $\hat{\beta}_j$, which are usually written as b_j . So, our predicted model becomes

$$\hat{y}_i = b_0 + b_1 x_{i1} + b_2 x_{i2} + \dots + b_p x_{ip}$$

The y has a hat on it to indicate this a predicted value of y rather than an actual y value. We compute these estimated parameters by finding the values that minimize the sum of the squared residuals. A residual is the difference between the actual y value and the predicted one. The residuals are written $e_i = y_i - \hat{y}_i$.

Consider an example of attempting to predict the divorce rate in Maine (in divorces per 1000 Mainers) using the margarine consumption of the United States (in pounds per person per year). This data set is called `divorce_margarine` and can be found in the `dslabs` R library.



This plot shows (perhaps unexpectedly!) a strong, positive, linear association between the two variables. So these variables would be good candidates for a regression model.

We can fit a regression model in **R** using the `lm()` function. Then we can display lots of useful output using the `summary()` function. Like so:

```
library(dslabs)
maine_mod <- lm(divorce_rate_maine ~ margarine_consumption_per_capita,
                data = divorce_margarine)
summary(maine_mod)
```

Call:

```
lm(formula = divorce_rate_maine ~ margarine_consumption_per_capita,
    data = divorce_margarine)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.05583	-0.01816	-0.01452	0.03601	0.04625

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	3.308626	0.048032	68.88	2.20e-12
margarine_consumption_per_capita	0.201386	0.008735	23.05	1.33e-08

(Intercept) ***

margarine_consumption_per_capita ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.03841 on 8 degrees of freedom

Multiple R-squared: 0.9852, Adjusted R-squared: 0.9833

F-statistic: 531.5 on 1 and 8 DF, p-value: 1.33e-08

This output tells us that the intercept of the model is 3.308626 and the slope is 0.201386. So, we can write the regression model

$$\hat{y} = 0.2014x + 3.309 \quad \text{or} \quad \hat{y} = 3.309 + 0.2014x$$

or, in the context of the problem:

$$\widehat{\text{divorce rate}} = 0.2014(\text{margarine}) + 3.309$$

We can then get a prediction for the divorce rate by plugging in a value for the margarine variable. For example, the predicted divorce rate in Maine when the margarine consumption in the US is 5 lbs per capita is $\text{divorce rate} = 0.2014(5) + 3.309 = 4.316$ per 1000 people. In **R**, we can do this using the `predict()` function.

```
predict(maine_mod, data.frame(margarine_consumption_per_capita = 5))
1
4.315556
```

We can even make a **confidence interval** for the average Y value using the `interval = "confidence"` argument. Or we can make a **prediction interval** for any Y value using the `interval = "prediction"` argument:

```
predict(maine_mod, data.frame(margarine_consumption_per_capita = 5),
        interval = "confidence")
      fit      lwr      upr
1 4.315556 4.286815 4.344298
```

```
predict(maine_mod, data.frame(margarine_consumption_per_capita = 5),
        interval = "prediction")
      fit      lwr      upr
1 4.315556 4.222435 4.408678
```

These intervals tell us each of the following:

- We are 95% confident that the average divorce rate in Maine for all years when the margarine consumption in the US is 5 lbs is between 4.29 and 4.34 per 1000 people.
- We are 95% confident that the divorce rate in Maine for any year when the margarine consumption in the US is 5 lbs is between 4.22 and 4.41 per 1000 people.

Intercept interpretation: The intercept tells us the predicted value of Y when X is equal to zero. In this case, we predict that the divorce rate in Maine will be 0.2014 per 1000 people when the per capita consumption of margarine in the US is zero lbs.

Slope interpretation: The slope is the predicted change in the Y variable when the X variable increases by 1 unit. In this case, as the per capita consumption of margarine in the US increases by 1 lb, the divorce rate in Maine is predicted to increase by 0.2014 per 1000 people. In multiple linear regression, there are more variables in the model, so we need to add the phrase “holding all other variables constant” at the end.

Some other important notes: This output is showing the results of two hypothesis tests, each with their own p-value. The p-values are shown in the `Pr(>|t|)` column. The one in the row for the intercept is testing the hypotheses

$$H_0 : \beta_0 = 0$$

$$H_A : \beta_0 \neq 0$$

In other words, it is testing if the true intercept is different than zero. The one in the row for the variable, `margarine_consumption_per_capita` in this case, is testing the hypotheses

$$H_0 : \beta_1 = 0$$

$$H_A : \beta_1 \neq 0$$

In other words, it is testing if the true slope is different than zero. Since the p-values are very small for both (notice the scientific notation: $2.20\text{e-}12 = 2.20 \times 10^{-12} = 0.0000000000022$), we have extremely strong evidence each parameter is not equal to zero.

Another statistic to pay attention to is the **Multiple R-squared**. This is the **coefficient of determination** and it tells us the proportion of the variation in the Y variable that is being explained by this model. Here, we can say that 98.52% of the variation in the divorce rate in Maine is explained by the per capita consumption of margarine in the US. This is a very high R^2 value and indicates this model fits the data extremely well.

Linear Regression Assumptions and Diagnostics

There are some assumptions that must be satisfied for the model to be fit and then for inference to be performed:

1. The means of the errors at a given value of X must be 0: $E(\varepsilon|x_i) = 0$ for all i . This indicates that the variables have a linear association. This assumption is the only one that is needed for prediction.
2. The variances of the errors at a given value of X must be equal: $\text{Var}(\varepsilon|x_i) = \sigma_\varepsilon^2$ for all i . This assumption is needed for inference.
3. The errors must follow a normal distribution: $\varepsilon \sim N$. This assumption is needed for inference.
4. The errors must be independent of each other. Having a random sample from the population ensures this. This assumption is needed for inference.

These assumptions are all about the true, population errors, which are unknown. So we never know for sure if these are met, but the reality is that these regression models work well if these are *reasonably* met. The way we assess them is by looking at some **diagnostic plots** like a **residual plot** and a **QQ plot**.

Residual Plots

Residual plots plot the residuals against the fitted values (the $\hat{y}$ values). These can be used to check the first and second assumptions listed above: the linearity and equal variance assumptions. We will want to plot the *externally studentized*, also known as the **jackknife**, residuals in this class since those are residuals that have been altered to be independent of each other approximately follow a t distribution. The jackknife residuals can be found in **R** using the `rstudent()` function. Ideally, the residual plots would show random scatter without any sort of discernible pattern and no fanning. It can be helpful to use `geom_smooth()` to plot a smoothed curve to the plot. This curve should be (mostly) horizontal at $y = 0$. A residual plot code template might look like this:

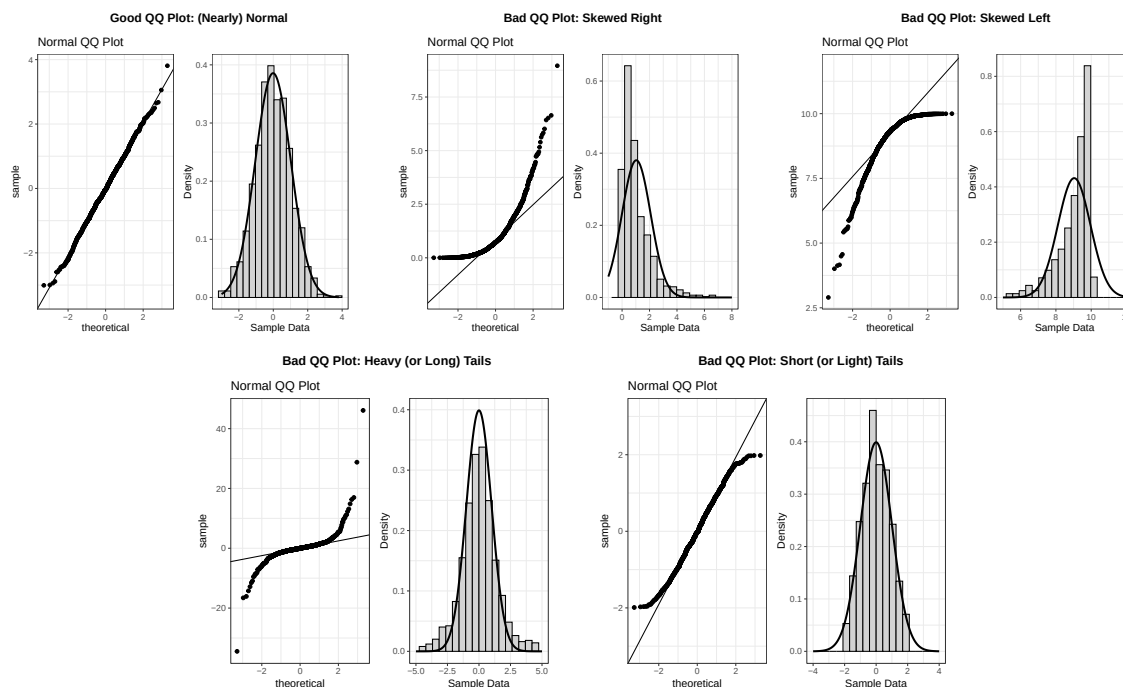
```
data.frame(resids = rstudent(regression_model),
           fitted = fitted.values(regression_model)) |>
  ggplot(aes(x = fitted, y = resids)) +
  geom_point() +
  geom_smooth(method = "loess", formula = "y ~ x", se = F) +
  labs(x = "Fitted Values", y = "Jackknife Residuals")
```

Ideally, the residual plots would show random scatter without any sort of discernible pattern and no fanning.



QQ Plots

QQ plots plot the sample quantiles against the theoretical quantiles. This can be used to check the third assumption above: the normality assumption. If the distribution of the sample (the residuals in our case) are approximately normal, then the points should fall along a straight line. Otherwise, they will deviate from the line. Regression inference is fairly robust against non-normality, so as long as the points are reasonably close to the line, then we will consider this assumption reasonably met.



We can make a QQ plot in **R** using the following code as a template:

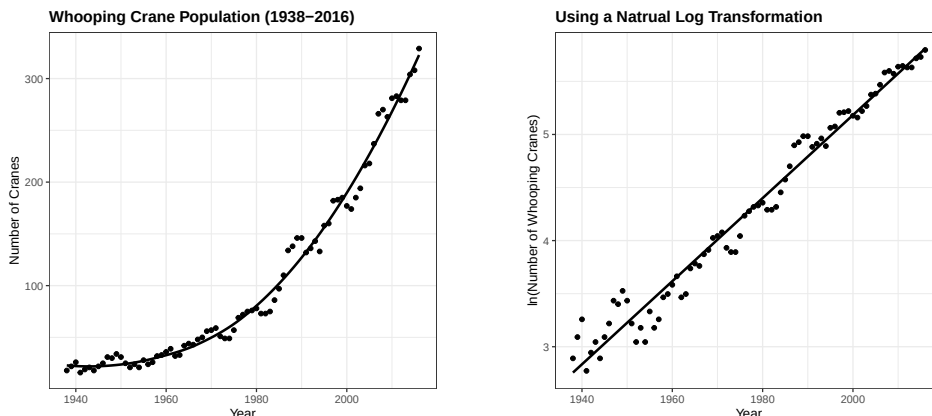
```
data.frame(resids = rstudent(regression_model)) |> # Studentized residuals
ggplot(aes(sample = resids)) +
  geom_qq_line() +
  geom_qq() +
  labs(title = "Normal QQ Plot")
```

What do we do if one or more of the assumptions does not seem to be reasonably satisfied? In that case, it is time for some **remedial measures**.

Regression Remedial Measures

Transform

Once thing we can do to potentially get the assumptions to be satisfied is to transform our data.



This is a very common technique, but is discussed at length in MATH 3150 and will not be a focus for us here.

Bootstrapping

We mentioned bootstrapping earlier in this class and we can rely on bootstrapping to obtain estimates of the standard errors when some of the assumptions are not reasonably met.

We will discuss two types of bootstrapping for regression:

1. Bootstrapping the residuals.
 - Used when the x values are fixed, the model provides an adequate fit, and the errors have constant variance.
2. Bootstrapping the cases.
 - Less sensitive to violations of any of the regression assumptions but problems can arise when there are outliers in the model.

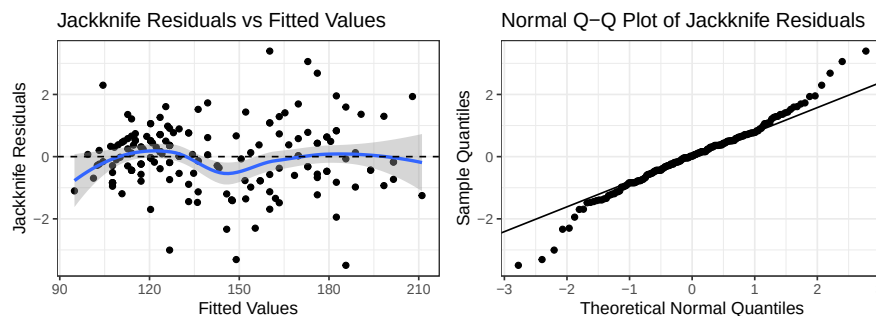
Bootstrapping the residuals

If we are reasonably assured that the model provides an adequate fit and the errors have constant variance, we can bootstrap the residuals to obtain standard errors for the regression intercept and slope(s). The process is as follows:

1. Fit a model to the data to obtain $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}$, $i = 1, \dots, n$.
2. Obtain the residuals for the model: $\hat{\epsilon}_i = y_i - \hat{y}_i$, $i = 1, \dots, n$.
3. Randomly sample the set of residuals with replacement to obtain $\hat{\epsilon}_i^*$, $i = 1, \dots, n$.
4. Create a set of pseudo-responses: $y_i^* = \hat{y}_i + \hat{\epsilon}_i^*$.
5. Fit a model to the pseudo-responses to obtain $\hat{y}_i^* = \hat{\beta}_0^* + \hat{\beta}_1^* x_{i1} + \cdots + \hat{\beta}_p^* x_{ip}$, $i = 1, \dots, n$.
6. Repeat steps 3-5 B times to obtain a distribution for each $\hat{\beta}_j^*$, $j = 0, \dots, p$.

Example:

Consider a data set where we are attempting to predict the ideal weights using the actual weights for males and females. Below are some plots for this model.



```
set.seed(2026)
wtmod <- lm(ideal ~ sex + actual, data = idealwt)
```

Call:

```
lm(formula = ideal ~ sex + actual, data = idealwt)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-30.6464	-5.0523	0.1796	4.7332	29.8019

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.64973	4.14729	9.078	<2e-16 ***
sexMale	23.93657	1.94410	12.312	<2e-16 ***
actual	0.63621	0.03059	20.797	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 9.131 on 179 degrees of freedom

Multiple R-squared: 0.9069, Adjusted R-squared: 0.9058

F-statistic: 871.3 on 2 and 179 DF, p-value: < 2.2e-16

```
confint(wtmod)
```

	2.5 %	97.5 %
(Intercept)	29.4658574	45.8336081
sexMale	20.1002659	27.7728749
actual	0.5758395	0.6965713

```
bootresids <- function(resids, i) {
  lm(I(wtmod$fitted + resids[i]) ~ sex + actual, data = idealwt)$coef
}
```

```
(boot_out1 <- bootstrapping(residuals(wtmod), bootresids, 5000))
```

A tibble: 3 x 5

term	sample_average	original_value	bias	std.error
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	37.7	37.6	0.0823	4.18
2 sexMale	24.0	23.9	0.0243	1.94
3 actual	0.636	0.636	-0.000593	0.0307

boot_ci(boot_out1)

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat1
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  29.4  45.8
2 basic           0.95  29.4  45.7
3 percent         0.95  29.6  45.9
4 bca             0.95  29.5  45.8

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

```

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat2
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  20.1  27.7
2 basic           0.95  20.0  27.7
3 percent         0.95  20.2  27.9
4 bca             0.95  20.2  27.9

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

```

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat3
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  0.577  0.697
2 basic           0.95  0.576  0.697
3 percent         0.95  0.575  0.696
4 bca             0.95  0.575  0.696

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

```

Bootstrapping the cases

If we are unsure that the model assumptions hold, we can bootstrap the cases to obtain standard errors for the regression intercept and slope(s). The process is as follows:

1. Sample n rows from the data frame with replacement.
2. Fit a model to the data using the sampled rows of the data frame to obtain $\hat{y}_i^* = \hat{\beta}_0^* + \hat{\beta}_1^* x_{i1} + \cdots + \hat{\beta}_p^* x_{ip}$, $i = 1, \dots, n$.
3. Repeat steps 1-2 B times to obtain a distribution for each $\hat{\beta}_j^*$, $j = 0, \dots, p$.

Example:

```

bootcases <- function(resids, i) {
  lm(ideal ~ sex + actual, data = idealwt[i,])$coef
}
boot_out2 <- bootstrapping(residuals(wtmod), bootcases, 5000)
boot_ci(boot_out2)

```

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat1
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  28.3  46.8
2 basic           0.95  27.9  46.6
3 percent         0.95  28.7  47.4
4 bca             0.95  28.7  47.5

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

```

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat2
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  19.2  28.6
2 basic           0.95  18.8  28.3
3 percent         0.95  19.5  29.0
4 bca             0.95  19.8  29.4

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

```

```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates
Info for stat3
# A tibble: 4 x 4
  interval_type conf_level lower upper
  <chr>          <dbl> <dbl> <dbl>
1 normal          0.95  0.564  0.710
2 basic           0.95  0.564  0.712
3 percent         0.95  0.560  0.708
4 bca             0.95  0.557  0.707

Calculations and Intervals on Original Scale
Some BCa intervals may be unstable

```

We can plot these bootstrap distributions using `plot(boot_out2)`, but that is omitted here.

Weighted Least Squares

An alternative to bootstrapping the cases when we have reason to believe the variances of the errors are not equal is to use **weighted least squares** to fit our regression model.

Matrix Formulation

We are used to regression models being written

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}$$

We can write this in matrix-vector form. If we define

$$\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n]_{n \times 1}^T, \quad \boldsymbol{\beta} = [\beta_0, \beta_1, \dots, \beta_p]_{(p+1) \times 1}^T, \quad \text{and}$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}_{n \times (p+1)}$$

then that equation for the i th row above can be written

$$\hat{\mathbf{y}}_i = \mathbf{X}_i \boldsymbol{\beta}$$

and then all n equations for the y values can be written

$$\hat{\mathbf{y}} = \mathbf{X} \boldsymbol{\beta}$$

In ordinary least squares (OLS) regression, we determine the estimates for the β values by computing

$$\hat{\boldsymbol{\beta}} = \mathbf{b} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Then we can compute standard errors for these estimates as long as the **linear regression assumptions** hold:

1. The residuals have zero mean (the relationship is linear).
2. The residuals have the same variance.
3. The residuals are normally distributed.
4. The residuals are independent.

If the homogeneity of variance assumption is violated, then the standard errors of the estimated β_j values are usually incorrect and that leads to confidence and prediction intervals being incorrect.

Standard Errors

We can obtain the standard errors of the parameter estimates by computing the variance-covariance matrix in the following manner:

$$\sigma_b^2 = \begin{bmatrix} \text{Var}(b_0) & \text{Cov}(b_0, b_1) & \dots & \text{Cov}(b_0, b_p) \\ \text{Cov}(b_1, b_0) & \text{Var}(b_1) & \dots & \text{Cov}(b_1, b_p) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(b_p, b_0) & \text{Cov}(b_p, b_1) & \dots & \text{Var}(b_p) \end{bmatrix}_{(p+1) \times (p+1)} = \sigma_\epsilon^2 (\mathbf{X}^T \mathbf{X})^{-1}$$

This means that the standard errors can be obtained by taking the square root of the diagonals of this matrix.

Consider this example of attempting to predict blood pressure using age.

```
library(math3190package)
bp <- blood_pressure |>
  rename(bp = diastolic_blood_pressure)
bp_mod <- lm(bp ~ age, data = bp)
X <- model.matrix(bp_mod)
var_matrix <- summary(bp_mod)$sigma^2 * solve(t(X) %*% X)
std_errors <- var_matrix |> diag() |> sqrt()
std_errors
(Intercept)      age
  3.99367376  0.09695116

# Compare to summary output
summary(bp_mod)
```

Call:

```
lm(formula = bp ~ age, data = bp)
```

Residuals:

Min	1Q	Median	3Q	Max
-16.4786	-5.7877	-0.0784	5.6117	19.7813

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	56.15693	3.99367	14.061	< 2e-16 ***
age	0.58003	0.09695	5.983	2.05e-07 ***

 Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

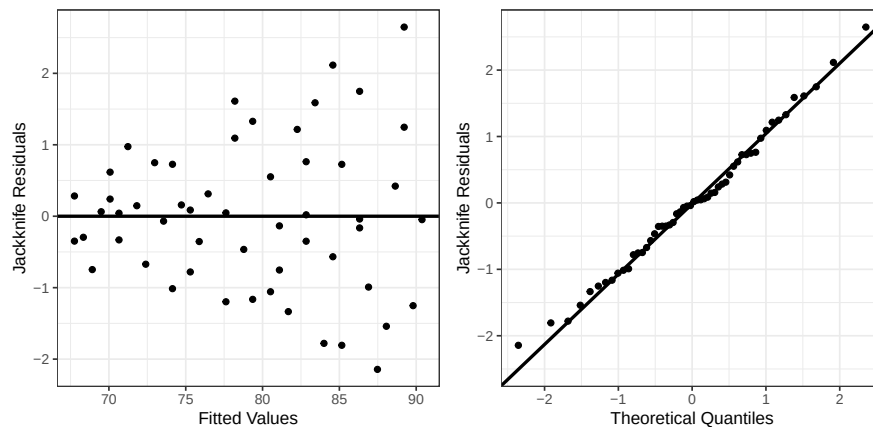
Residual standard error: 8.146 on 52 degrees of freedom

Multiple R-squared: 0.4077, Adjusted R-squared: 0.3963

F-statistic: 35.79 on 1 and 52 DF, p-value: 2.05e-07

Original Diagnostic Plots

Below is what the diagnostic plots looks like for this example:



In order to adjust so the estimates and standard errors are not as effected by the points that

have large residuals, we can **weight** each observation. We will define a vector of weights as $\mathbf{w} = [w_1, w_2, \dots, w_n]^T$ and a matrix of the weights as

$$\mathbf{W} = \begin{bmatrix} w_1 & 0 & \dots & 0 \\ 0 & w_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & w_n \end{bmatrix}$$

We can then obtain the weighted estimates of the coefficients, called $\mathbf{b}_w$, by computing

$$\mathbf{b}_w = (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \mathbf{y}$$

with variance-covariance matrix

$$\sigma_b^2 = \sigma_\epsilon^2 \cdot (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1}.$$

Selecting the Weights

In ordinary least squares (OLS), we assume $\text{Var}(\epsilon_i) = \sigma_\epsilon^2$ for all i . In WLS, we instead allow for each error to have its own variance. So, we have $\text{Var}(\epsilon_i) = \sigma_i^2 = \sigma_\epsilon^2 / w_i$. Our best guess for σ_ϵ^2 is the mean square error (MSE). That is what we typically denote as s_ϵ^2 .

Now we need to determine the best values for the weights. First, we must obtain an estimate for the variance function for the residuals. This will depend on the residual plot(s), but the most common situation for using weighted least squares is when the residuals plotted against the fitted values has a fan shape. In this scenario, we can estimate the weights by:

1. Fit an ordinary least squares model (and verify that a residual plot against the fitted values has a fan shape).
2. Fit a new linear model by regressing the absolute value of the residuals on the fitted values. That is, use $|\hat{\epsilon}|$ as y and $\hat{y}$ as x .
3. Let the weights be the reciprocals of the squares of the fitted values of that second model.

Assessing the Adequacy of the Weights

Each of the following should be true when using weighted least squares:

1. The estimates of the β values using WLS should be very close to the estimates of the β values using OLS.
2. The weighted mean square error (MSE_w) should be close to 1.
3. The residual plot of the jackknife residuals should have constant variance.

Also note the following things about WLS:

1. R^2 has no real meaning in weighted least squares and should not be considered. We should use the R^2 from the OLS model.
2. Externally studentized (jackknife) residuals take into account the weights, so they should be used in residual plots for model diagnostics.
3. For QQ plots, use the unstudentized residuals. Either the raw or standardized are fine.

```
bp_mod <- lm(bp ~ age, data = bp)
e_mod <- lm(abs(bp_mod$residuals) ~ bp_mod$fitted.values)
```

```
w <- 1/e_mod$fitted.values^2
```

```
bp_mod_w <- lm(bp ~ age, data = bp, weights = w)
```

```
summary(bp_mod_w)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	55.56577	2.52092	22.042	< 2e-16 ***
age	0.59634	0.07924	7.526	7.19e-10 ***

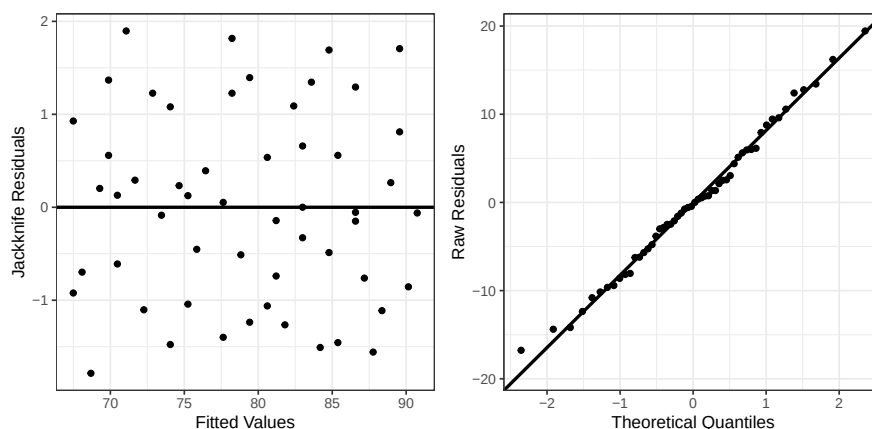
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.213 on 52 degrees of freedom

Multiple R-squared: 0.5214, Adjusted R-squared: 0.5122

F-statistic: 56.64 on 1 and 52 DF, p-value: 7.187e-10

Weighted Diagnostic Plots



Confidence Intervals

We can easily obtain confidence intervals for the β 's by using the `confint()` function. The variance-covariance matrix is now $\sigma_\epsilon^2(\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}$.

To obtain confidence intervals for predictions, that is, a confidence interval for $E(y|\mathbf{x}^*)$, we first consider what the CI looks like in the OLS case. For obtaining the $(1 - \alpha) \cdot 100\%$ confidence interval for $E(y|\mathbf{x}^*)$ at a specific row vector $\mathbf{x}^* = [1 \ x_1^* \ \dots \ x_n^*]$ is given by

$$\hat{y}|\mathbf{x}^* \pm t_{\alpha/2} s_\epsilon \sqrt{\mathbf{x}^*(\mathbf{X}^T\mathbf{X})^{-1}(\mathbf{x}^*)^T}$$

For weighted least squares, we just need to insert the weight matrix:

$$\hat{y}|\mathbf{x}^* \pm t_{\alpha/2} s_w \sqrt{\mathbf{x}^*(\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}(\mathbf{x}^*)^T}$$

where s_w is the square root of the weighted MSE.

An example for obtaining the bounds for blood pressure at age 50:

```

W <- diag(w)
n <- nrow(bp)
alpha <- 0.05
x_star <- matrix(c(1, 50), nrow = 1)
yhat <- predict(bp_mod, data.frame(age = 50))
yhat_w <- predict(bp_mod_w, data.frame(age = 50))
se <- summary(bp_mod)$sigma
se_w <- summary(bp_mod_w)$sigma
# OLS Bounds
yhat - qt(1-alpha/2, n-2) * se * sqrt(x_star %*% solve(t(X) %*% X) %*% t(x_star))
      [,1]
[1,] 82.14817
yhat + qt(1-alpha/2, n-2) * se * sqrt(x_star %*% solve(t(X) %*% X) %*% t(x_star))
      [,1]
[1,] 88.16877
# Verify using the predict function
predict(bp_mod, data.frame(age = 50), interval = "confidence")
      fit      lwr      upr
1 85.15847 82.14817 88.16877
# WLS Bounds - Confidence Interval
yhat_w - qt(1 - alpha/2, n-2) * se_w * sqrt(
  x_star %*% solve(t(X) %*% W %*% X) %*% t(x_star)
)
      [,1]
[1,] 81.84106
yhat_w + qt(1 - alpha/2, n-2) * se_w * sqrt(
  x_star %*% solve(t(X) %*% W %*% X) %*% t(x_star)
)
      [,1]
[1,] 88.92464
# Verify using the predict function
predict(bp_mod_w, data.frame(age = 50), interval = "confidence")
      fit      lwr      upr
1 85.38285 81.84106 88.92464

```

The prediction interval for OLS is computed with

$$\hat{y}|\mathbf{x}^* \pm t_{\alpha/2} s_{\epsilon} \sqrt{1 + \mathbf{x}^*(\mathbf{X}^T \mathbf{X})^{-1}(\mathbf{x}^*)^T}$$

For weighted least squares, we just need to change that 1 to $\frac{1}{w^*}$ over the weight for that particular $\mathbf{x}^*$:

$$\hat{y}|\mathbf{x}^* \pm t_{\alpha/2} s_w \sqrt{\frac{1}{w^*} + \mathbf{x}^*(\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1}(\mathbf{x}^*)^T}$$

where w^* is the weight for the value we are predicting.

```

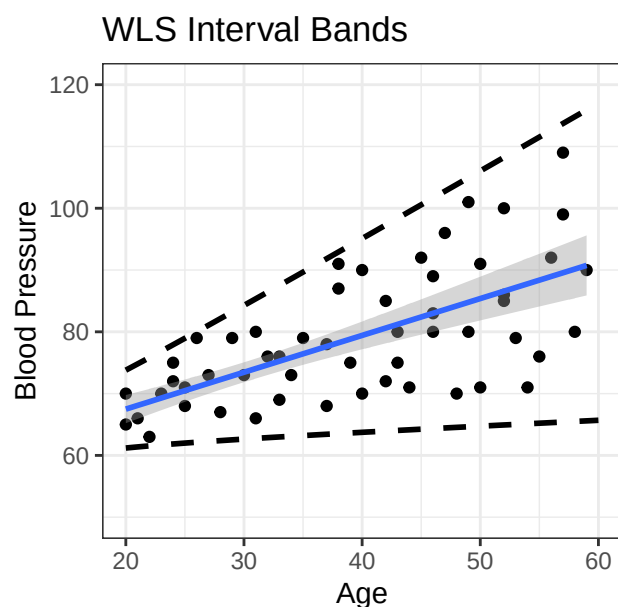
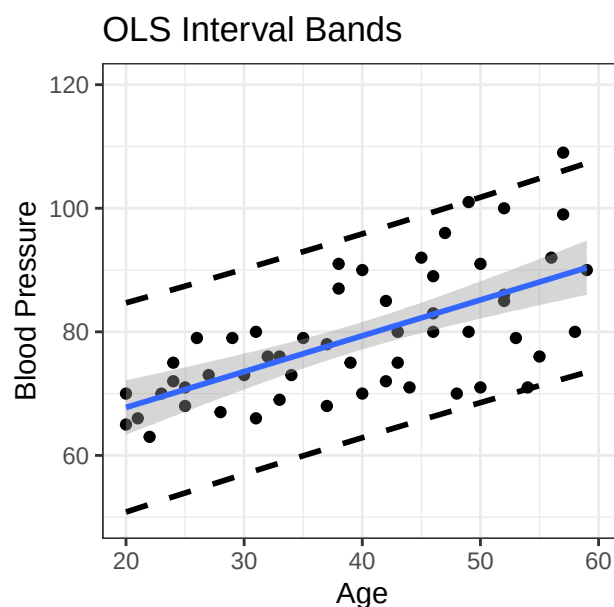
# Prediction interval OLS
lwr_bound <- yhat - qt(1 - alpha/2, n-2) * se * sqrt(
  1 + x_star %>% solve(t(X) %>% X) %>% t(x_star)
)
upr_bound <- yhat + qt(1 - alpha/2, n-2) * se * sqrt(
  1 + x_star %>% solve(t(X) %>% X) %>% t(x_star)
)
c(lwr_bound, upr_bound)
[1] 68.53795 101.77899

predict(bp_mod, data.frame(age = 50), interval = "prediction")
      fit      lwr      upr
1 85.15847 68.53795 101.779

# Prediction interval WLS
# Get new weight for x = 50:
new_fit <- predict(bp_mod, data.frame(age = 50))
new_weight <- 1/predict(e_mod, data.frame(fitted = new_fit))^2
# Construct Intervals
lwr_bound <- yhat_w - qt(1-alpha/2, n-2) * se_w * sqrt(
  1/new_weight + x_star %>% solve(t(X) %>% W %>% X) %>% t(x_star)
)
upr_bound <- yhat_w + qt(1-alpha/2, n-2) * se_w * sqrt(
  1/new_weight + x_star %>% solve(t(X) %>% W %>% X) %>% t(x_star)
)
c(lwr_bound, upr_bound)
[1] 64.72992 106.03578

predict(bp_mod_w, data.frame(age=50), interval="prediction", weight = new_weight)
      fit      lwr      upr
1 85.38285 64.72992 106.0358

```

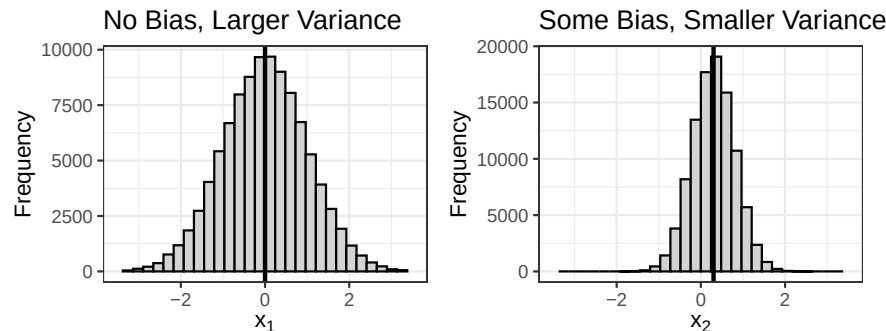


Regularization – Ridge, LASSO, and Elastic Net

Bias-Variance Trade Off

In statistical estimation, there is often a trade off between bias and variance. Many statistical methods, like least squares regression, guarantee that the estimate is **unbiased**, which means that the sampling distribution of the statistic is centered at the parameter it is estimating.

However, these unbiased estimators will often have more uncertainty, a higher variance, than some biased estimators. When this is the case, the biased estimator that is more precise may be preferred.

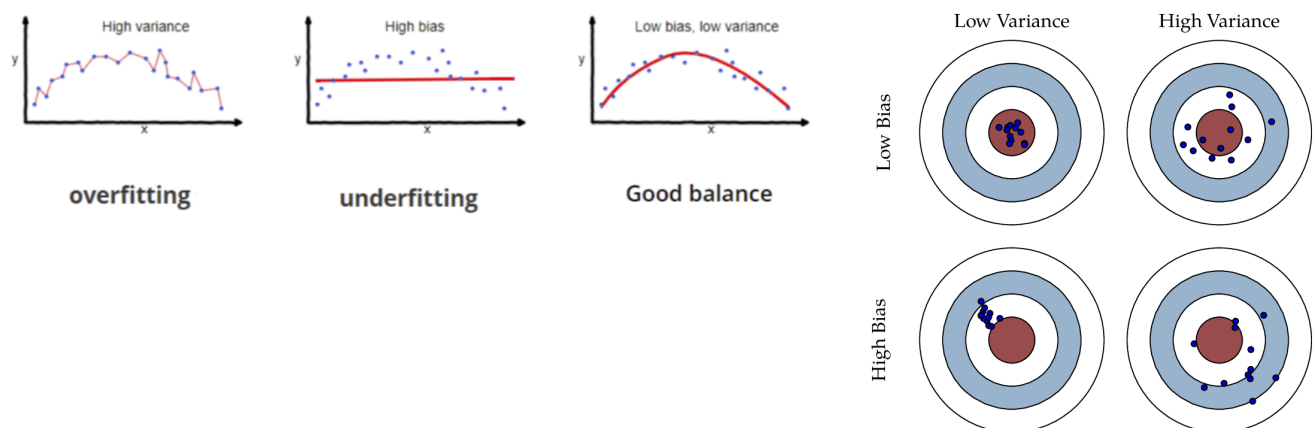


In statistics and machine learning, the **bias–variance tradeoff** is the property of a model that the variance of the parameter estimated across samples can be reduced by increasing the bias in the estimated parameters.

The **bias–variance dilemma** or **bias–variance problem** is the conflict in trying to simultaneously minimize these two sources of error that prevent supervised learning algorithms from generalizing beyond their training set. (Source: Wikipedia)

The **bias** is an error from faulty assumptions or misspecification of the learning algorithm. High bias can cause an algorithm to miss the relevant relations between features and target outputs (underfitting).

The **variance** is an error from sensitivity to small fluctuations in the training set. High variance may result from an algorithm modeling the random noise in the training data (overfitting).



The bias–variance trade off is a central problem in supervised learning (where the y values are known). Ideally, one wants to choose a model that both accurately captures the regularities in its

training data, but also generalizes well to unseen data.

Unfortunately, it is typically impossible to do both simultaneously. High-variance learning methods may be able to represent their training set well but are at risk of overfitting to noisy or unrepresentative training data. In contrast, algorithms with high bias typically produce simpler models that may fail to capture important regularities (i.e. underfit) in the data.

Regularization in Machine Learning

In regression analysis, the features are estimated using coefficients while modeling. In small sample sizes or noisy data coefficient estimates could be anecdotally incorrect (e.g., overfitting) or inaccurate.

If the estimates can be restricted, penalized, or shrunk towards zero, then the impact of insignificant features might be reduced and would prevent models from high variance with a stable fit.¹

Regularization is the most used technique to penalize complex models in machine learning, it is deployed for reducing overfitting (or, contracting generalization errors) by putting small network weights into the model (adding a small amount of bias). Also, it enhances the performance of models for new inputs.

Examples of regularization in machine learning, include:

- K-means: Restricting the segments for avoiding redundant groups.
- Neural networks: Confining the complexity (weights) of a model.
- Random forests: Reducing the depths of tree and branches (new features).

Ridge Regression

Ridge regression **regularizes** (shrinks) coefficients by imposing a penalty on their size. The ridge coefficients minimize a penalized sum of squared error:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\},$$

where $\lambda \geq 0$ is a parameter that controls the shrinkage. The larger the value of λ the more shrinkage (towards 0). Or in matrix form, ridge regression minimizes:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \left\{ (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) + \lambda \beta^T \beta \right\}.$$

With a little work, the ridge regression solution can be shown to be:

$$\hat{\beta}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_N)^{-1} \mathbf{X}^T \mathbf{y}$$

Penalty Terms

The regularization for ridge regression, $\lambda \beta^T \beta$, is usually denoted as an **L2 regularization** or **L2 penalty**, as it adds a penalty which is equal to the square of the magnitude of coefficients. Both Ridge regression and Support Vector Machines (SVMs) implement this method. L2 regularization can deal with multicollinearity problems (independent variables are highly correlated) through constricting the coefficient while keeping all the variables in a model. However, L2 regularization is not an effective method for selecting relevant predictors (or removing redundant parameters). We will use a **L1 regularization** for this purpose.

¹Adapted from: <https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning>

LASSO Regression

LASSO (Least Absolute Shrinkage and Selection Operator) regression also **regularizes** coefficients by imposing a penalty on their size, but it uses an **L1** penalty. The LASSO coefficients minimize the following cost function:

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \alpha \sum_{j=1}^p |\beta_j| \right\},$$

where $\alpha \geq 0$ is a parameter that controls the shrinkage. The larger the value of α the more shrinkage (towards 0).

Notice the similarity to the ridge regression problem: the L2 ridge penalty $\sum_{j=1}^p \beta_j^2$ is replaced by the L1 lasso penalty $\sum_{j=1}^p |\beta_j|$.

This latter constraint makes the solutions nonlinear in the y_i , and there is no closed form expression for the LASSO as was the case in ridge regression. Because of the nature of the constraint, making α sufficiently small will cause some of the coefficients to be exactly zero. Thus the LASSO does a kind of continuous subset selection, or conducts a **variable selection**.

LASSO vs Ridge Regression

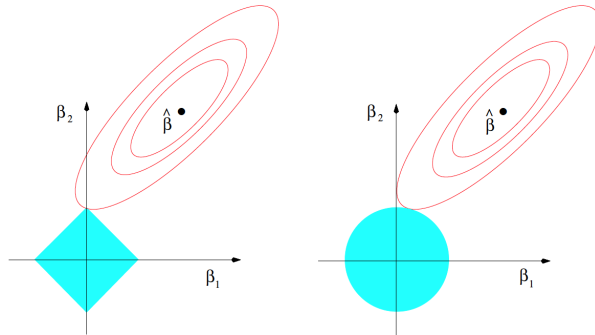


FIGURE 3.11. Estimation picture for the lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions $|\beta_1| + |\beta_2| \leq t$ and $\beta_1^2 + \beta_2^2 \leq t^2$, respectively, while the red ellipses are the contours of the least squares error function.

S.No	L1 Regularization	L2 Regularization
1	Panelizes the sum of absolute value of weights.	penalizes the sum of square weights.
2	It has a sparse solution.	It has a non-sparse solution.
3	It gives multiple solutions.	It has only one solution.
4	Constructed in feature selection.	No feature selection.
5	Robust to outliers.	Not robust to outliers.
6	It generates simple and interpretable models.	It gives more accurate predictions when the output variable is the function of whole input variables.
7	Unable to learn complex data patterns.	Able to learn complex data patterns.
8	Computationally inefficient over non-sparse conditions.	Computationally efficient because of having analytical solutions.

Left image is from [Elements of Statistical Learning](#), Hastie, Tibshirani, Friedman, by Springer.

Right image is from <https://www.analyticssteps.com/blogs/l2-and-l1-regularization-machine-learning>.

Elastic Net Regularization

Which should we choose? Ridge or LASSO? Well, why do we have to choose?!

Instead we could use the **Elastic Net** that minimizes:

$$\hat{\beta}_{\text{elastic net}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \alpha_1 \sum_{j=1}^p |\beta_j| + \alpha_2 \sum_{j=1}^p \beta_j^2 \right\},$$

for some $\alpha_1 \geq 0$ and $\alpha_2 \geq 0$.

The quadratic penalty term makes the loss function strongly convex, and it therefore has a unique minimum. The elastic net method includes Ridge regression, LASSO, and OLS by setting either $\alpha_1 = 0$, $\alpha_2 = 0$, or both to 0.

Regression Regularization in R: glmnet

We can use the `glmnet` and `glmnetUtils` package to apply regularization in R:

```
install.packages("glmnet")
install.packages("glmnetUtils")
```

The default model used in the package is the “least squares” regression model and `glmnet` actually optimizes:

$$\hat{\beta}_{\text{elastic net}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \left(\alpha \sum_{j=1}^p |\beta_j| + \frac{(1-\alpha)}{2} \sum_{j=1}^p \beta_j^2 \right) \right\},$$

with $\alpha = 1$ as a default (so LASSO!).

As an example, let’s look at a dataset for predicting body fat of a person using a variety of variables. This dataset can be found in the `faraway` package.

```
library(glmnet) |> suppressPackageStartupMessages()
library(glmnetUtils) |> suppressPackageStartupMessages()
library(faraway) |> suppressPackageStartupMessages()
fat_mod <- lm(
  brozek ~ age + weight + height + neck + chest +
    abdom + hip + thigh + knee + ankle + biceps +
    forearm + wrist, data = fat
) # Fits an OLS model
```

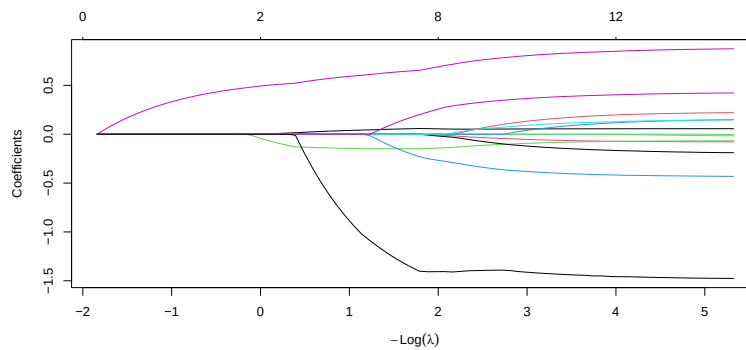
We fit the model using the most basic call to `glmnet`. $\alpha = 1$ by default, but let’s also specified it here.

```
fit_lasso <- glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1
) # Fits a LASSO
```

`fit_lasso` is now an object of class `glmnet` that contains all the relevant information of the fitted model for further use. There are various functions that work well with this class ob object such as `plot`, `print`, `coef` and `predict`.

We can visualize the size of the coefficients for different λ values by using the `plot()` function:

```
plot(fit_lasso, "lambda")
```



We can obtain the model coefficients at one or more λ values within the range of the sequence:

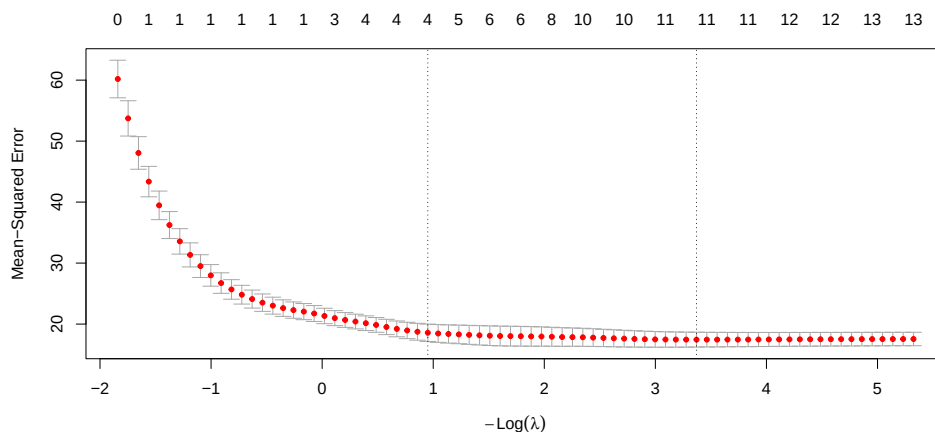
```
coef(fit_lasso, s = 0.1) # s is the value for a "specific" lambda
```

```
14 x 1 sparse Matrix of class "dgCMatrix"
```

```
      s=0.1
(Intercept) -7.53281619
age          0.05183206
weight      -0.02504372
height      -0.12561764
neck        -0.31157363
chest       .
abdom       0.73717170
hip         -0.04875740
thigh       0.02570643
knee        .
ankle       .
biceps      0.03422877
forearm     0.30749547
wrist       -1.40042018
```

The function `glmnet()` returns a sequence of models for the users to choose from. **Cross-validation** is perhaps the simplest and most widely used method to select a model. `cv.glmnet()` is the main function to do cross-validation here, along with various supporting methods such as plotting and prediction.

```
set.seed(2026)
cvfit_lasso <- cv.glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1
)
plot(cvfit_lasso)
```



Coefficients Using Minimum Lambda

We can get the value of λ_{min} and the model coefficients:

```
cvfit_lasso$lambda.min
[1] 0.03438172

coef(cvfit_lasso, s = "lambda.min")
14 x 1 sparse Matrix of class "dgCMatrix"
      lambda.min
(Intercept) -13.34535803
age          0.05478517
weight      -0.06316555
height      -0.08317641
neck        -0.39947252
chest        .
abdom        0.82585758
hip         -0.14325642
thigh        0.16479917
knee         .
ankle        0.07804134
biceps       0.10961077
forearm      0.38403630
wrist       -1.43316868
```

Coefficients Using Lambda within 1 SE of Min

The output of the `cv.glmnet()` function also gives us the largest λ that is within 1 SE of the λ that gave the smallest MSE. This λ_{1se} value penalizes the coefficients more and therefore chooses a simpler model.

```
cvfit_lasso$lambda.1se
[1] 0.3862181
```

```
coef(cvfit_lasso, s = "lambda.1se")
14 x 1 sparse Matrix of class "dgCMatrix"
      lambda.1se
(Intercept) -11.8103049
age          0.0371891
weight       .
height      -0.1445500
neck         .
chest        .
abdom        0.5880006
hip          .
thigh        .
knee         .
ankle        .
biceps       .
forearm      .
wrist        -0.8339864
```

Fitting Model with Selected λ

Then we can fit our model using the λ value that we choose.

```
fit_min <- glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1,
  lambda = cvfit_lasso$lambda.min
)
c(fit_min$lambda, fit_min$dev.ratio)
[1] 0.03438172 0.74755316
```

```
fit_1se <- glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1,
  lambda = cvfit_lasso$lambda.1se
)
c(fit_1se$lambda, fit_1se$dev.ratio)
[1] 0.3862181 0.7066715
```

LASSO vs Stepwise Selection

Since LASSO is primarily used for model selection, it makes sense to compare it to another common model selection tool: the step wise selection.

```
# step(fat_mod) # Using AIC
# step(fat_mod, k = log(nrow(fat))) # Using BIC
# coef(fit_min) # LASSO
# coef(fit_1se) # LASSO
```

Method	Model
Stepwise AIC	$-20.1 + 0.059 \cdot \text{age} - 0.084 \cdot \text{weight} - 0.432 \cdot \text{neck} + 0.877 \cdot \text{abdom}$ $-0.186 \cdot \text{hip} + 0.286 \cdot \text{thigh} + 0.483 \cdot \text{forearm} - 1.40 \cdot \text{wrist}$
Stepwise BIC	$-31.3 - 0.126 \cdot \text{weight} + 0.921 \cdot \text{abdom} + 0.446 \cdot \text{forearm} - 1.39 \cdot \text{wrist}$
LASSO (min λ)	$-13.3 + 0.055 \cdot \text{age} - 0.063 \cdot \text{weight} - 0.083 \cdot \text{height} - 0.399 \cdot \text{neck}$ $+0.826 \cdot \text{abdom} - 0.143 \cdot \text{hip} + 0.165 \cdot \text{thigh} + 0.078 \cdot \text{ankle}$ $+0.110 \cdot \text{biceps} + 0.384 \cdot \text{forearm} - 1.43 \cdot \text{wrist}$
LASSO (1 SE λ)	$-11.8 + 0.037 \cdot \text{age} - 0.144 \cdot \text{height} + 0.588 \cdot \text{abdom} - 0.835 \cdot \text{wrist}$

Note that the variables selected by the LASSO are **not unique** due to the randomness associated with choosing λ . The cross-validation randomly selects groups from the data to fit the model and that can lead to a different “best” λ value. For example, if our seed is set to a different number, we obtain a different result. The λ can change in ridge regression as well, but the number of variables will not change with ridge.

```
set.seed(2026)
cv.glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1
)$lambda.min
[1] 0.03438172
```

```
set.seed(1)
cv.glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 1
)$lambda.min
[1] 0.03132734
```

Ridge Regression Example

Now let's use ridge regression to get estimates for the coefficients. We will set $\alpha = 0$ here. This does not reduce the number of predictors at all.

```
set.seed(2026)
cvfit_ridge <- cv.glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 0
) # Set alpha = 0 for ridge regression
fit_ridge <- glmnet(
  brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat, alpha = 0,
```

```
lambda = cvfit_ridge$lambda.min
)
coef(fit_ridge)
14 x 1 sparse Matrix of class "dgCMatrix"
      s0
(Intercept) -9.969782963
age          0.100897615
weight      -0.006623162
height      -0.162911219
neck        -0.314865758
chest        0.121310803
abdom        0.489012499
hip         -0.005349405
thigh        0.178795904
knee        -0.039658185
ankle       -0.010673900
biceps       0.071735533
forearm      0.294566945
wrist       -1.514325320
```

Ridge Regression VIF Reduction

One common use of ridge regression is eliminating some problems caused by multicollinearity, which occurs when the predictors are highly correlated with each other. When this is the case, the slopes are difficult to interpret and the standard errors in the slope estimates get inflated. Let's check this in our model for body fat.

```
options(width = 70)
library(car) # vif() function is in the car package
vif(fat_mod)
      age    weight    height    neck    chest    abdom    hip
2.250450 33.509320  1.674591  4.324463  9.460877 11.767073 14.796520
      thigh    knee    ankle    biceps    forearm    wrist
7.777865  4.612147  1.907961  3.619744  2.192492  3.377515
```

The VIF tells us how much the variance of the estimates of the slopes are inflated. For example, the variance of the **abdom** variable is 11.767 times higher (so the SE is about 3.43 times larger) than it would be if multicollinearity was not present.

```
summary(fat_mod)
Coefficients:
      Estimate Std. Error t value Pr(>|t|)
(Intercept) -15.29255    16.06992  -0.952  0.34225
age           0.05679     0.02996   1.895  0.05929 .
weight       -0.08031     0.04958  -1.620  0.10660
height       -0.06460     0.08893  -0.726  0.46830
neck         -0.43754     0.21533  -2.032  0.04327 *
chest        -0.02360     0.09184  -0.257  0.79740
abdom         0.88543     0.08008  11.057 < 2e-16 ***
hip          -0.19842     0.13516  -1.468  0.14341
```


thigh	0.23190	0.13372	1.734	0.08418	.
knee	-0.01168	0.22414	-0.052	0.95850	
ankle	0.16354	0.20514	0.797	0.42614	
biceps	0.15280	0.15851	0.964	0.33605	
forearm	0.43049	0.18445	2.334	0.02044	*
wrist	-1.47654	0.49552	-2.980	0.00318	**

While there is not a closed-form expression for the standard error of the ridge regression estimates, we can obtain estimated standard errors using either bootstrapping or Bayesian methods (we must wait until Notes 9 for Bayesian!). Let's look at an example using bootstrapping:

```
set.seed(2026)
nboot <- 1000
coefs <- matrix(rep(0, nboot*14), nrow = nboot)
for(i in 1:nboot) {
  index <- sample(1:nrow(fat), nrow(fat), replace = T)
  mod <- cv.glmnet(
    brozek ~ age + weight + height + neck + chest + abdom + hip + thigh +
    knee + ankle + biceps + forearm + wrist, data = fat[index,], alpha = 0
  )
  coefs[i,] <- coef(mod, s = "lambda.min")[1:14]
}
c(sd(coefs[,2]), sd(coefs[,3]), sd(coefs[,7]))
[1] 0.02148759 0.01561954 0.04080476
```

The standard errors for the slope of **age** decreased from 0.02996 to 0.02149 (a factor of about 1.39, so the variance decreased by about a factor of 1.94). The SE for the slope of **weight** decreased from 0.04958 to 0.01562 (a factor of about 3.17, variance by a factor of about 10.07), and the SE for the slope of **abdom** decreased from 0.08008 to 0.04080 (a factor of about 1.96, variance by a factor of about 3.85). We get these reductions without even removing any variables from the model. Of course, these estimates are now biased. By exactly how much is hard to determine.

Alternative way to get VIFs

Another way to obtain the VIF values in OLS is by computing the diagonals of the inverse of the correlation matrix for the predictors: $\mathbf{r}_{xx}^{-1}$.

In ridge regression, the VIF values can be computed by taking the diagonal elements of

$$(\mathbf{r}_{xx} - \lambda \mathbf{I})^{-1} \mathbf{r}_{xx} (\mathbf{r}_{xx} - \lambda \mathbf{I})^{-1}.$$

The original VIF values:

```
r_xx <- fat |> select(age, weight, height, neck, chest, abdom, hip,
  thigh, knee, ankle, biceps, forearm, wrist) |>
  cor()
# Alternatively, can do
# r_xx <- cor(model.matrix(fat_mod)[-1]) # Remove the first column since all 1's
r_xx |> solve() |> diag() |> round(5)
```

age	weight	height	neck	chest	abdom	hip
2.25045	33.50932	1.67459	4.32446	9.46088	11.76707	14.79652
thigh	knee	ankle	biceps	forearm	wrist	

```
7.77786 4.61215 1.90796 3.61974 2.19249 3.37751
```

In our case for this toy example, a good λ value was 0.6294. The reduced ridge regression VIF values:

```
diag(
  solve(r_xx + 0.6294*diag(rep(1, 13))) %*%
    r_xx %*%
    solve(r_xx + 0.6294*diag(rep(1, 13)))
) |> round(5)
      age weight height   neck   chest   abdom    hip   thigh
0.33414 0.08150 0.36442 0.29041 0.21550 0.18218 0.16728 0.21672
      knee  ankle biceps forearm  wrist
0.28443 0.36689 0.30987 0.35652 0.31358
```

We actually obtain VIF values less than 1. This can happen under ridge regression and indicates that the multicollinearity issue has likely been resolved. There are some articles that propose different ways to calculate VIF in ridge regression, however, like this one [here](#).

glmnet in the caret package

While we were able to obtain an “optimal” λ value using the `cv.glmnet()` function, it did not have an option to optimize α as well. If all we care about are predictions and are not trying to use LASSO for model selection or ridge regression for VIF reduction, then choosing α via cross validation can make sense.

```
library(caret) |> suppressPackageStartupMessages()
set.seed(2026)
search_grid <- expand.grid(alpha = seq(0, 1, length = 50),
                           lambda=seq(0, 1, by = 0.1))
train_glmnet <- train(
  brozek ~ age + weight + height + neck + chest +
    + abdom + hip + thigh + knee + ankle + biceps +
    forearm + wrist, data = fat, method = "glmnet",
  tuneGrid = search_grid, trControl = trainControl(method = "cv", number = 5)
)
train_glmnet$bestTune
      alpha lambda
210 0.3877551      0
```

The best λ was chosen to be zero, which likely means are search grid steps were too large. Try again with a smaller number in the `by` argument.

```
library(caret) |> suppressPackageStartupMessages()
set.seed(2026)
search_grid <- expand.grid(alpha = seq(0, 1, length = 50),
                           lambda=seq(0, 0.1, by = 0.001))
train_glmnet <- train(
  brozek ~ age + weight + height + neck + chest +
    + abdom + hip + thigh + knee + ankle + biceps +
    forearm + wrist, data = fat, method = "glmnet",
```

```
tuneGrid = search_grid, trControl = trainControl(method = "cv", number = 5)
)
train_glmnet$bestTune
      alpha lambda
4881 0.9795918 0.032
```

This found that an elastic net with $\alpha = 0.9796$ and $\lambda = 0.032$ is best in terms of minimizing the MSE. α is close to 1 here, so this is close to a LASSO.

Example: Logistic Regression Elastic Net

We can do regularization using ridge regression, LASSO, or elastic net in other models as well, like logistic regression.

Logistic regression is a widely-used model when the response is binary. Suppose the response variable y takes values $\{0,1\}$. We model

$$P(y = 1|\mathbf{X}_i) = \frac{e^{\mathbf{X}_i\beta}}{1 + e^{\mathbf{X}_i\beta}},$$

which can be written in the following form:

$$\log\left(\frac{P(y = 1|\mathbf{X}_i)}{P(y = 0|\mathbf{X}_i)}\right) = \mathbf{X}_i\beta,$$

the so-called “logistic” or log-odds transformation. We seek to minimize the following loss function:

$$\arg \min_{\beta} \left\{ -\frac{1}{n} \sum_{i=1}^n y_i \log(\mathbf{X}_i\beta) - \log(1 + e^{\mathbf{X}_i\beta}) + \lambda \left(\alpha \sum_{j=1}^p |\beta_j| + \frac{(1-\alpha)}{2} \sum_{j=1}^p \beta_j^2 \right) \right\},$$

where the left hand side is the loss function for standard logistic regression, and the right hand side is the elastic net regularization.

Logistic regression is often plagued with degeneracies when $p > n$ and exhibits wild behavior even when n is close to p ; the elastic net penalty alleviates these issues, and regularizes and selects variables as well.

Using the example data set from the glmnet package:

```
library(glmnet)
data(BinomialExample)
logistic_df <- data.frame(y = BinomialExample$y,
                          as.data.frame(BinomialExample$x)) # Currently a matrix
names(logistic_df) <- c("y", paste0("x", 1:30))
```

Let’s try to fit a logistic regression to this dataset with $n = 100$ and $p = 31$ (30 predictor variables and only 100 data points).

```
logistic_fit <- glm(y ~ ., data = logistic_df, family = "binomial")
Warning: glm.fit: algorithm did not converge
Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
summary(logistic_fit)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	7.815e+00	5.947e+04	0.000	1.000
x1	1.536e+01	4.586e+04	0.000	1.000
x2	4.219e+01	5.145e+04	0.001	0.999
x3	-1.483e+01	5.276e+04	0.000	1.000
x4	-2.831e+01	3.652e+04	-0.001	0.999
x5	-4.968e+00	5.440e+04	0.000	1.000
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.

Here we are getting warnings about the algorithm not converging because the fitted probabilities were exactly 0 or 1. This is an issue, likely because we have so many predictor variables.

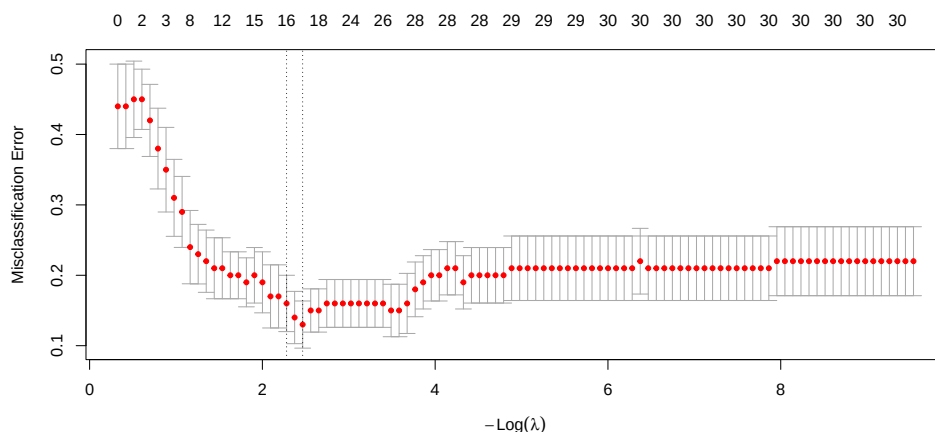
Now let's try with the elastic net with $\alpha = 1/3$ (equal weight on the L1 and L2 penalty). Why is $\alpha = 1/2$ not an equal weight?

Set family option to “binomial” in the `glmnet` or `cv.glmnet` functions. The code below uses misclassification error as the criterion for 10-fold cross-validation:

```
set.seed(2026)
cvfit <- cv.glmnet(y ~ ., family = "binomial", alpha = 1/3,
                  data = logistic_df, type.measure = "class")
```

Now we can plot the cross-validation results and find the “best” λ_{min} :

```
plot(cvfit)
```



```
cvfit$lambda.min
[1] 0.08489859
```

```
logistic_glmnet_fit <- glmnet(y ~ ., family = "binomial", alpha = 1/3,
                             data = logistic_df, lambda = cvfit$lambda.min)
coef(logistic_glmnet_fit)
```

```

31 x 1 sparse Matrix of class "dgCMatrix"
              s0
(Intercept)  0.2307632187
x1           .
x2           0.2593622891
x3          -0.1882223855
x4          -0.5861095408
x5          -0.1643935161
x6          -0.3938875709
x7           .
x8          -0.2670402697
x9           0.2345566373
x10          -0.6131022751
x11          -0.0672497953
x12          -0.0009538166
x13           .
x14           .
x15           .
x16           0.0557649394
x17           .
x18           .
x19           .
x20           .
x21           .
x22           0.1360558390
x23           0.1722437787
x24           .
x25           0.2883397048
x26          -0.2155427412
x27           .
x28           0.0868855495
x29          -0.1174446662
x30           .

```

Poisson Regression

When doing regression analysis in Applied Statistics, we discussed linear regression with a continuous response variable, logistic regression where the response variable is binary, and multinomial logistic regression where the response variable is nominal with several (more than two) categories.

We did not discuss the best way to handle a discrete, quantitative response variable. If this is the case, the possible values range from 0 to infinity, and large counts are rare, then **Poisson regression** is most appropriate.

Recall the Poisson distribution from MATH 1040, 1190, or 3700:

$$P(Y = y) = f(y) = \frac{e^{-\lambda} \lambda^y}{y!} \quad \text{for } y = 0, 1, 2, \dots$$

with $E[Y] = \text{Var}(Y) = \lambda$. So, the variance increases as the mean of Y increases.

We often use capital letters (like Y) to denote the variable name and lowercase letters (like y) to denote values of that variable.

Poisson Link Function

Like when doing logistic regression, we need a link function for Poisson regression. In logistic regression, the most common link function is the logit function: $\log\left(\frac{P(Y=1|\mathbf{X}_i)}{1 - P(Y=1|\mathbf{X}_i)}\right)$ and when

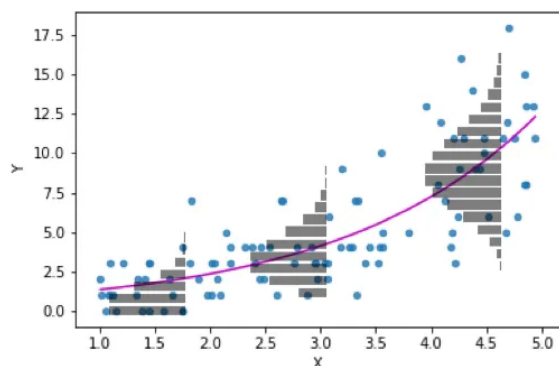
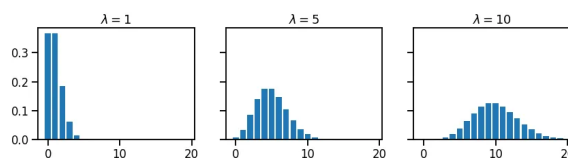
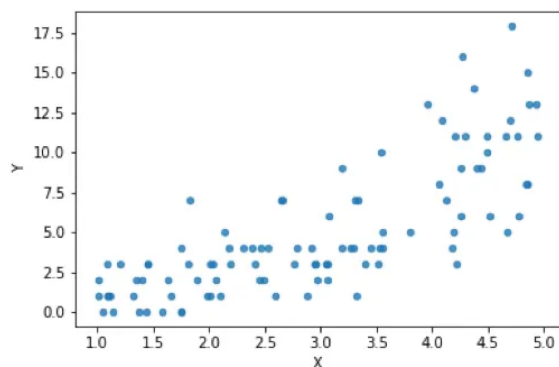
using that function, the model becomes linear. That is, $\log\left(\frac{P(Y=1|\mathbf{X}_i)}{1 - P(Y=1|\mathbf{X}_i)}\right) = \mathbf{X}_i\beta$.

The most common link function for Poisson regression is just the log function. Typically, we just write “log” to indicate the natural log function. That is,

$$\log(\widehat{E[Y|\mathbf{X}_i]}) = \mathbf{X}_i\beta$$

where $\mathbf{X}_i$ is the i th row of the model matrix $\mathbf{X}$. Of course, the expected value of Y is λ , so this model is really for predicting λ at certain values of X : $\log(\hat{\lambda}_i|x_i) = \beta_0 + \beta_1 x_i$ or $\hat{\lambda}_i|x_i = e^{\beta_0 + \beta_1 x_i}$ in the SLR case.

With the Poisson regression, we model the data by thinking about Y as following a Poisson distribution with a different mean at every value of x . So, if we begin with a dataset that looks like the one in the image below, which is fairly common for Poisson regression, then we can imagine a different Poisson distribution at each x value.



Poisson Regression Assumptions

1. **Poisson Response:** The response variable is conditionally a count per unit of time or space, described by a Poisson distribution.
2. **Independence:** The observations must be independent of one another.
3. **Mean = Variance:** By definition, the mean of a Poisson random variable must be equal to its variance.
4. **Linearity:** The log of the mean rate, $\log(\lambda)$, must be a linear function of x .

Finding Parameter Estimates

The parameters, $\beta_0, \beta_1, \dots, \beta_p$ are not estimated using least squares. Instead, like for logistic regression, these are estimated using **maximum likelihood estimation** (MLE), which we discussed earlier in the class. Let's apply the MLE technique to the Poisson regression case:

$$\begin{aligned} L(\beta) &= \prod_{i=1}^n f_i(y_i) = f_1(y_1) \cdots f_n(y_n) \\ &= \frac{e^{-\lambda_1} \lambda_1^{y_1}}{y_1!} \cdots \frac{e^{-\lambda_n} \lambda_n^{y_n}}{y_n!} \\ &= \frac{\prod_{i=1}^n \lambda_i^{y_i} \cdot \exp\{-\sum_{i=1}^n \lambda_i\}}{\prod_{i=1}^n y_i!} \end{aligned}$$

Now, it is common to take the log of the likelihood to make the actual maximizing easier:

$$\begin{aligned} \ln(L(\beta)) &= \ell(\beta) = \ln\left(\frac{\prod_{i=1}^n \lambda_i^{y_i} \cdot \exp\{-\sum_{i=1}^n \lambda_i\}}{\prod_{i=1}^n y_i!}\right) \\ &= \ln\left(\prod_{i=1}^n \lambda_i^{y_i}\right) + \ln\left(\exp\left\{-\sum_{i=1}^n \lambda_i\right\}\right) - \ln\left(\prod_{i=1}^n y_i!\right) \\ &= \sum_{i=1}^n y_i \ln(\lambda_i) - \sum_{i=1}^n \lambda_i - \sum_{i=1}^n \ln(y_i!) \end{aligned}$$

Now we can substitute for λ_i . That is, $\lambda_i = E(Y|\mathbf{X}_i) = \exp(\mathbf{X}_i\beta)$:

$$\begin{aligned} \ell(\beta) &= \sum_{i=1}^n y_i \ln(\lambda_i) - \sum_{i=1}^n \lambda_i - \sum_{i=1}^n \ln(y_i!) \\ &= \sum_{i=1}^n y_i \ln(\exp(\mathbf{X}_i\beta)) - \sum_{i=1}^n \exp(\mathbf{X}_i\beta) - \sum_{i=1}^n \ln(y_i!) \\ &= \sum_{i=1}^n y_i (\mathbf{X}_i\beta) - \sum_{i=1}^n \exp(\mathbf{X}_i\beta) - \sum_{i=1}^n \ln(y_i!) \end{aligned}$$

Now, to maximize this, like in calculus, we take the derivative with respect to one of the parameters (one of the β 's) and set the equation equal to zero. Let's do an example in the SLR case where

$$\mathbf{X}_i\boldsymbol{\beta} = \beta_0 + \beta_1 x_i.$$

$$\begin{aligned}\ell(\boldsymbol{\beta}) &= \sum_{i=1}^n y_i(\mathbf{X}_i\boldsymbol{\beta}) - \sum_{i=1}^n \exp(\mathbf{X}_i\boldsymbol{\beta}) - \sum_{i=1}^n \ln(y_i!) \\ &= \sum_{i=1}^n y_i(\beta_0 + \beta_1 x_i) - \sum_{i=1}^n \exp(\beta_0 + \beta_1 x_i) - \sum_{i=1}^n \ln(y_i!)\end{aligned}$$

Take the derivative with respect to β_0 and set it equal to zero:

$$\frac{\partial \ell}{\partial \beta_0} = \sum_{i=1}^n y_i - \sum_{i=1}^n \exp(\beta_0 + \beta_1 x_i) = 0$$

Take the derivative with respect to β_1 and set it equal to zero:

$$\frac{\partial \ell}{\partial \beta_1} = \sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \exp(\beta_0 + \beta_1 x_i) = 0$$

These equations need to be solved numerically (unlike ordinary least squares), so they do not have a closed-form solution. We discussed using a function like `optim()` in **R** to solve optimization problems like this, but that only works if we have x and y values. So, let's simulate a little toy example:

```
set.seed(2026)
x <- rnorm(5) # Simulate 5 random values for x
# Simulate 5 random values for y (with a couple different means)
y <- c(rpois(3, 2), rpois(2, 6))
first_func <- function(beta) {
  sum(y) - sum(exp(beta[1] + beta[2] * x))
}
second_func <- function(beta) {
  sum(x * y) - sum(x * exp(beta[1] + beta[2] * x))
}
optim_func <- function(beta) {
  # This will be minimized when both functions are zero
  abs(first_func(beta)) + abs(second_func(beta))
}
optim(c(1, 0), optim_func)

$par
[1] 1.0837225 -0.4531598

$value
[1] 2.597732e-07

$counts
function gradient
      133         NA

$convergence
[1] 0

$message
NULL
```


Fitting the Poisson Model in R

In **R**, we can use the `glm()` function to fit a Poisson model. Like we do with logistic regression, we need to specify the family and the link function. By default, the link function in the “poisson” family is the log, so we will keep it as the default.

```
glm(y ~ x, data = poisson_df, family = "poisson")
```

So applying this to the little example from before:

```
mod <- glm(y ~ x, family = "poisson") # Fit the Poisson model
summary(mod)
```

Call:

```
glm(formula = y ~ x, family = "poisson")
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	1.0837	0.2918	3.713	0.000205	***
x	-0.4532	0.4227	-1.072	0.283648	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 15.624 on 4 degrees of freedom
 Residual deviance: 14.468 on 3 degrees of freedom
 AIC: 30.779

Number of Fisher Scoring iterations: 6

Now let's do a much more substantial example:

Illustrative Example - Lumber

The Miller Lumber Company is a large retailer of lumber and paint, as well as of plumbing, electrical, and other household supplies. During a representative two-week period, in-store surveys were conducted and addresses of customers were obtained. The addresses were then used to identify the metropolitan area census tracts in which the customers reside. At the end of the survey period, the total number of customers who visited the store from each census tract within a 10-mile radius was determined and relevant demographic information for each tract (average income, number of housing units, etc.) was obtained. Several other variables expected to be related to customer counts were constructed from maps, including distance from census tract to nearest competitor and distance to store.

We will try to predict the number of customers from a Census tract using two variables: the distance the track is to the Miller Lumber Company store (quantitative in miles) and the distance from a competitor's store (categorical with categories “<2”, “2-4”, or “4+” miles.)

```
library(math3190package) |> suppressPackageStartupMessages()
lumber <- lumber_company |>
  mutate(
    competitor_dist = factor(
```

```

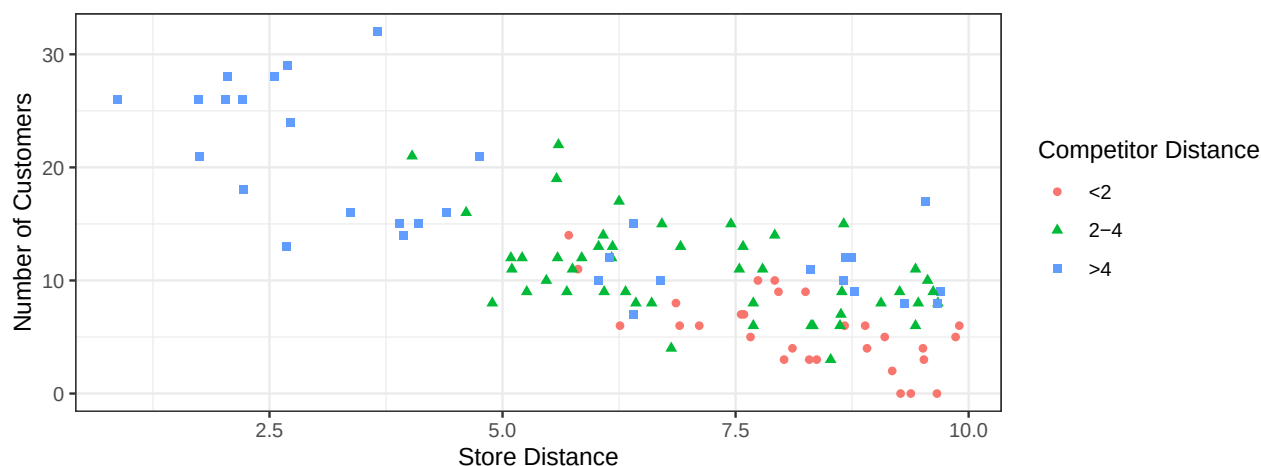
    case_when(competitor_distance <= 2 ~ "<2",
              competitor_distance > 2 & competitor_distance <= 4 ~ "2-4",
              competitor_distance > 4 ~ ">4"),
    levels = c("<2", "2-4", ">4")
  )
)
lumber_mod <- glm(customers ~ store_distance * competitor_dist,
                  data = lumber, family = "poisson")

```

```

ggplot(lumber, aes(x = store_distance, y = customers,
                  shape = competitor_dist,
                  color = competitor_dist)) +
  geom_point() +
  labs(x = "Store Distance", y = "Number of Customers",
       shape = "Competitor Distance", color = "Competitor Distance") +
  theme_bw()

```



```
summary(lumber_mod)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	4.32561	0.50344	8.592	< 2e-16	***
store_distance	-0.32659	0.06425	-5.083	3.71e-07	***
competitor_dist2-4	-1.12808	0.54137	-2.084	0.03718	*
competitor_dist>4	-0.89883	0.50992	-1.763	0.07795	.
store_distance:competitor_dist2-4	0.20830	0.07034	2.961	0.00306	**
store_distance:competitor_dist>4	0.19871	0.06623	3.000	0.00270	**

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 422.22 on 109 degrees of freedom
 Residual deviance: 132.35 on 104 degrees of freedom
 AIC: 588.39

We will get one equation for each category of the categorical variable. Letting x_i be the i th value of `store_distance`, we obtain:

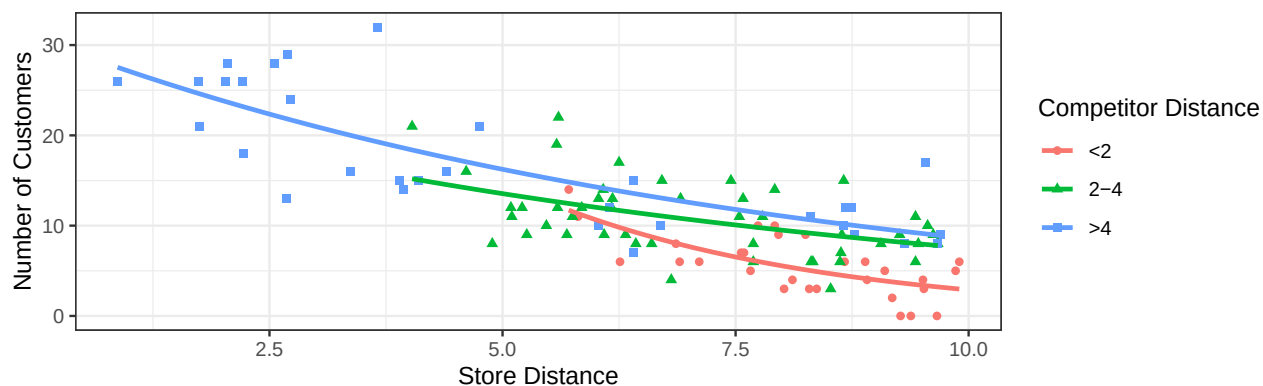
Model 1 (<2 Group): $\ln(\hat{\lambda}_i) = 4.32561 - 0.32659 \cdot x_i$

Model 2 (2-4 Group): $\ln(\hat{\lambda}_i) = 3.19753 - 0.11829 \cdot x_i$

Model 3 (4+ Group): $\ln(\hat{\lambda}_i) = 3.42678 - 0.12788 \cdot x_i$

We can use the `geom_smooth()` function to plot the models curves.

```
ggplot(lumber, aes(x = store_distance, y = customers,
                  shape = competitor_dist,
                  color = competitor_dist)) +
  geom_point() +
  labs(x = "Store Distance", y = "Number of Customers",
       shape = "Competitor Distance", color = "Competitor Distance") +
  geom_smooth(method = "glm", method.args = list(family = "poisson"),
             formula = "y ~ x", se = F) + theme_bw()
```



Interpreting the Model Coefficients

The interpretation of the coefficients here is quite similar to interpreting the coefficients of a log-transformed linear regression model.

Intercept: The predicted mean of the Poisson distribution when $x = 0$ is $\exp(\hat{\beta}_0)$.

Slope: The predicted change in the mean of the Poisson distribution when x increases by one unit is $\exp(\hat{\beta}_1)$.

Confidence Intervals for the Model Coefficients

We can obtain confidence intervals for the intercept and slope(s) using normal approximations as long as our sample size is fairly large:

$$\left(\hat{\beta}_i - z_{\alpha/2} \cdot SE(\hat{\beta}_i), \hat{\beta}_i + z_{\alpha/2} \cdot SE(\hat{\beta}_i)\right)$$

Then we can exponentiate these to get the confidence intervals back on the original scale:

$$\left(\exp\left\{\hat{\beta}_i - z_{\alpha/2} \cdot SE(\hat{\beta}_i)\right\}, \exp\left\{\hat{\beta}_i + z_{\alpha/2} \cdot SE(\hat{\beta}_i)\right\}\right)$$

Using the `confint()` function in **R** uses what is called the “profile likelihood” method for computing these intervals, which we will not discuss here, so the results will be a bit different. Note that the **MASS** package must be installed for this to work.

Lumber Example Confidence Intervals

For the “store_distance” variable, the β_1 estimate is -0.32659 with a standard error of 0.06425 . We can obtain an approximate 95% interval using

```
-0.32659 - qnorm(1-0.05/2) * 0.06425
[1] -0.4525177
```

```
-0.32659 + qnorm(1-0.05/2) * 0.06425
[1] -0.2006623
```

So the interval is $-0.453 < \beta_1 < -0.201$.

Using the `confint()` function:

```
confint(lumber_mod)[2,]
Waiting for profiling to be done...
      2.5 %      97.5 %
-0.4522654 -0.2001689
```

We obtain an interval of $-0.452 < \beta_1 < -0.200$.

Confidence Intervals for the Mean Response

We can obtain an approximate confidence interval for $E(Y_i|\mathbf{x}_i)$ using the `predict()` function with the options `type = "response"` and `se.fit = T`. This won't create for CI for us, though. We can approximate using a normal distribution. This approximation is better for larger values of λ_i .

```
lumber_pred <- predict(lumber_mod,
                      data.frame(store_distance = 5,
                                competitor_dist = "2-4"),
                      se.fit = T)
exp(lumber_pred$fit - 1.96 * lumber_pred$se.fit)
exp(lumber_pred$fit + 1.96 * lumber_pred$se.fit)

11.87221; 15.45597
```

So, the interval is $11.87 < E(Y^*|\mathbf{x}^*) < 15.46$ or $11.87 < \lambda^*|\mathbf{x}^* < 15.46$.

Prediction Intervals

These confidence intervals are for the mean Poisson response at a given row vector $\mathbf{x}$. Then, to get predictions for the actual count, we can use percentiles for Poisson distribution.

For the lower bound of a $(1 - \alpha) \times 100\%$ confidence interval, we can use the $\alpha/2 \times 100$ th percentile using the lower bound of the CI for the mean.

For the upper bound of a $(1 - \alpha) \times 100\%$ confidence interval, we can use the $(1 - \alpha/2) \times 100$ th percentile using the upper bound of the CI for the mean.

```
qpois(0.025, 11.87221) # 2.5th percentile using lower mean
[1] 6
```

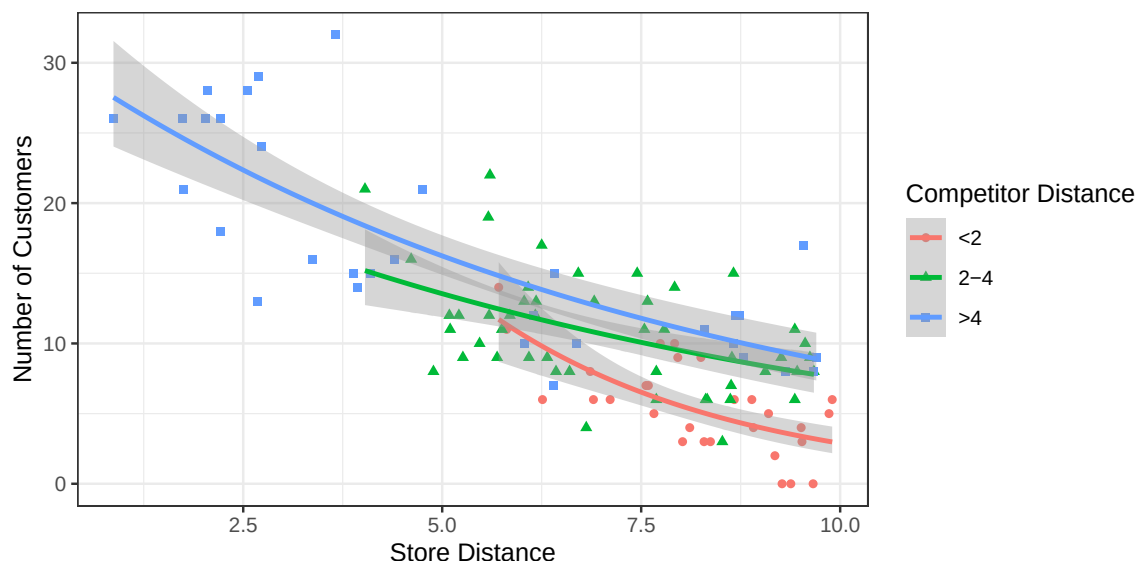
```
qpois(0.975, 15.45597) # 97.5th percentile using upper mean
[1] 24
```

That gives an interval of $6 < y^* | \mathbf{x}^* < 24$.

Lumber Model Plot with CI

We can plot the CI for the mean with the `geom_smooth()` function as well. These intervals are constructed using the profile likelihood method rather than normal approximations, but they are similar.

```
ggplot(lumber,
  aes(x = store_distance, y = customers,
    shape = competitor_dist, color = competitor_dist)) +
  geom_point() +
  labs(x = "Store Distance",
    y = "Number of Customers",
    shape = "Competitor Distance",
    color = "Competitor Distance") +
  geom_smooth(method = "glm",
    method.args = list(family = "poisson"),
    formula = "y ~ x", se = T) +
  theme_bw()
```



Poisson Residuals

There are several different types of residuals for a Poisson regression model. The first is called the **Pearson residual**, which is computed using

$$\hat{\epsilon}_i^P = \frac{y_i - \hat{y}_i}{\sqrt{\hat{y}_i}}$$

where $\hat{y}_i = \exp(\mathbf{X}_i \hat{\boldsymbol{\beta}}) = \hat{\lambda}_i$.

Then there is the **deviance residual**, which is computed using

$$\hat{\epsilon}_i^D = \text{sgn}(y_i - \hat{y}_i) \sqrt{2(y_i \cdot \ln(y_i/\hat{y}_i) - y_i + \hat{y}_i)}$$

where sgn is the sign function that is positive when its input is positive and negative otherwise. Note that when $y_i = 0$, we take $y_i \cdot \ln(y_i/\hat{y}_i)$ to also equal 0.

It can be shown that both of these residuals are approximately normally distributed.

Other Poisson Regression Functions in R

To obtain the fitted values on the original data scale, we can either use the

```
fitted.values()
```

function or use

```
predict(model, type = "response")
```

If we just use

```
predict(model)
```

it will return the linear predictors. To get back to the original scale, we would need to exponentiate those values.

To obtain the Pearson residuals, we can use

```
residuals(model, type = "pearson")
```

The deviance residuals are the default, so they be obtained using either

```
residuals(model)
```

```
residuals(model, type = "deviance")
```

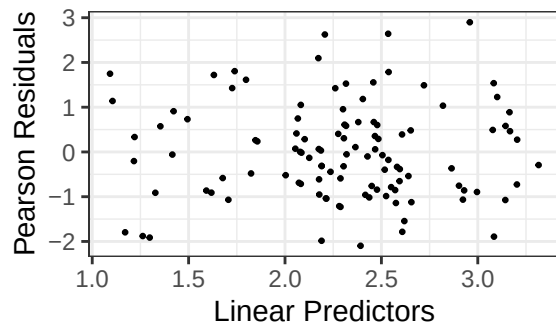
Poisson Regression Diagnostics

We can perform **diagnostics** for Poisson regression using hypothesis tests and plots. Each of the following are helpful:

1. A residual plot of the Pearson residuals vs the predicted values. These residuals should appear randomly spread.
2. A QQ-plot of the Pearson or deviance residuals. These should be approximately normally distributed.
3. A deviance goodness-of-fit (GOF) test.
4. A check for overdispersion.

Poisson Regression Diagnostics - Residual Plot

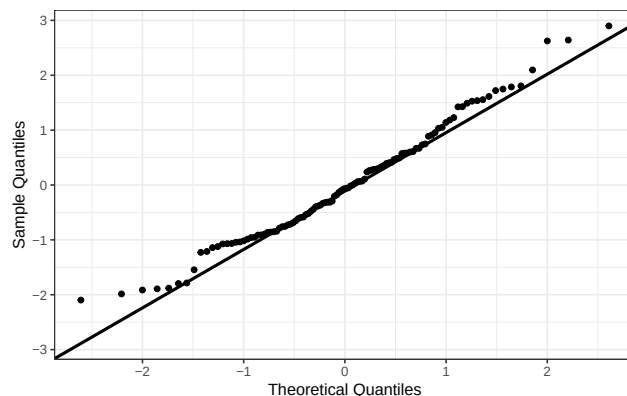
```
p_resids <- residuals(lumber_mod, type = "pearson")
resid_df <- data.frame(p_resids,
                      linear_pred = lumber_mod$linear.predictors)
ggplot(resid_df, aes(x = linear_pred, y = p_resids)) +
  geom_point(size = 0.5) +
  labs(x = "Linear Predictors", y = "Pearson Residuals") +
  theme_bw()
```



Poisson Regression Diagnostics - QQ Plot

We can use a **QQ plot** to check if the Pearson residuals are approximately normal. This can be done with the `qqnorm()` and `qqline()` function in base **R** or the `geom_qq()` and `geom_qq_line()` in `ggplot2`. The closer to the line, the better.

```
ggplot(resid_df, aes(sample = p_resids)) +
  geom_qq() +
  geom_qq_line(linewidth = 1) +
  labs(x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_bw()
```



Poisson Regression Diagnostics - GOF Test

A **deviance goodness-of-fit (GOF)** test tests the hypotheses

H_0 : The model is correctly specified.

H_A : The model is incorrectly specified.

The test statistic is

$$D = 2 \sum_{i=1}^n (y_i \ln(y/\hat{y}_i) - (y_i - \hat{y}_i)) = \sum_{i=1}^n (\hat{\epsilon}_i^D)^2,$$

the sum of the squared deviance residuals. This statistic approximately follows a χ^2 distribution with $n - p$ degrees of freedom when the predicting means are not too small. It is also given in the output of the Poisson model as the “Residual deviance”.

If this statistic is large relative to the degrees of freedom, we have evidence that the model does not fit well, and we would reject the null hypothesis. Therefore, this is a right-tailed test.

```
1 - pchisq(lumber_mod$deviance, df = lumber_mod$df.residual)
[1] 0.03169943
```

We have evidence here that this model does not fit well. In this case, that is likely because the model is **overdispersed**. We can alleviate this issue by adding more important variables. If we add more variables, we lose evidence the model is poorly fitting.

```
lumber_mod2 <- glm(customers ~ store_distance + competitor_dist +
  average_income + average_age + housing_units,
  data = lumber, family = "poisson")
1 - pchisq(lumber_mod2$deviance, df = lumber_mod2$df.residual)
[1] 0.2755826
```

Poisson Regression Diagnostics - Overdispersion

One important property of a Poisson distribution is that $E(Y) = \lambda$ and $Var(Y) = \lambda$. That is, both the mean and the variance are equal. **Overdispersion** occurs when the variance of a model is greater than its mean.

- Overdispersion is caused by a positive correlation between responses or by too much variation between response probabilities or counts. Overdispersion also arises when there are violations in the distributional assumptions of the data as well as when the data are prone to it (i.e., when earlier events cause or influence the existence of subsequent events).
- Overdispersion is an issue because it may cause standard errors of the estimates to be underestimated (i.e., a variable may appear to be a significant predictor when it is in fact not significant).
- A model may be overdispersed if the value of the sum of the squared Pearson residuals divided by the degrees of freedom (df) is greater than 1.0. This quotient is called the dispersion. Small amounts of overdispersion are of little concern; however, if the dispersion statistic is greater than 1.25 for moderate-sized models, then a correction may be warranted. Models with large numbers of observations may be overdispersed with a dispersion statistic of 1.05.
- We can alleviate the overdispersion issue by adding other predictors, transforming the count data, or **changing models**.

If we are using Poisson regression in its proper context, then the most common way to alleviate overdispersion is to change from a Poisson model to a **quasi-Poisson model**.

Quasi-Poisson Regression

The quasi-Poisson distribution is very similar to the Poisson, except it allows for the mean and variance to differ:

$$E[Y] = \lambda$$

$$Var(Y) = \phi\lambda$$

where ϕ is known as the dispersion parameter that is estimated from the data.

In particular, $\phi = \frac{1}{n-p} \sum_{i=1}^n (\hat{\epsilon}_i^P)^2$, the sum of the squared Pearson residuals divided by the residual degrees of freedom.

Quasi-Poisson Example

In our example, let's check the estimate for ϕ :

```
sum(residuals(lumber_mod, type = "pearson")^2)
[1] 124.7268

lumber_mod$df.residual # n - p = 110 - 6
[1] 104

phi <- sum(residuals(lumber_mod, type = "pearson")^2)/lumber_mod$df.residual
phi
[1] 1.199297
```

The dispersion estimate of 1.199 is not above the 1.25 cutoff, but it fairly large. Let's fit a new model using a "quasi-poisson" family.

```
lumber_mod_quasi <- glm(
  formula = customers ~ store_distance * competitor_dist,
  family = "quasipoisson", data = lumber)
```

Let's compare the output.

```
summary(lumber_mod)

Coefficients:
                                Estimate Std. Error z value Pr(>|z|)
(Intercept)                4.32561      0.50344    8.592 < 2e-16 ***
store_distance             -0.32659      0.06425   -5.083 3.71e-07 ***
competitor_dist2-4         -1.12808      0.54137   -2.084 0.03718 *
competitor_dist>4          -0.89883      0.50992   -1.763 0.07795 .
store_distance:competitor_dist2-4 0.20830      0.07034    2.961 0.00306 **
store_distance:competitor_dist>4  0.19871      0.06623    3.000 0.00270 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(Dispersion parameter for poisson family taken to be 1)

```
Null deviance: 422.22  on 109  degrees of freedom
Residual deviance: 132.35  on 104  degrees of freedom
AIC: 588.39
```

Quasi-Poisson Example - Quasi-Poisson Output

```
summary(lumber_mod_quasi)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	4.32561	0.55133	7.846	3.99e-12	***
store_distance	-0.32659	0.07036	-4.642	1.01e-05	***
competitor_dist2-4	-1.12808	0.59287	-1.903	0.05984	.
competitor_dist>4	-0.89883	0.55843	-1.610	0.11052	
store_distance:competitor_dist2-4	0.20830	0.07703	2.704	0.00800	**
store_distance:competitor_dist>4	0.19871	0.07253	2.740	0.00724	**

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for quasipoisson family taken to be 1.199297)

Null deviance: 422.22 on 109 degrees of freedom
 Residual deviance: 132.35 on 104 degrees of freedom
 AIC: NA

Some things to notice:

- The parameter estimates are the same.
- We estimate the standard errors of the $\hat{\beta}_i$ coefficients by taking the SE estimates in the regular Poisson regression model and multiplying them by $\sqrt{\phi}$.
- These estimates for the quasi-Poisson approximately follow a t distribution rather than a normal distribution.
- AIC is not defined for the quasi-Poisson family.

An alternative to using the quasi-Poisson is to use **negative binomial regression**. Poisson regression is actually just a special case of negative binomial regression and the latter has more than one parameter to allow for the mean and variance to differ.

Just for Fun - Poisson Bootstrap

The bootstrap never (or at least rarely) lies! Compare these standard errors to the Poisson and quasi-Poisson ones.

```
nsims <- 5000
betas <- matrix(rep(0, 6*nsims), nrow = nsims)
for(i in 1:nsims) {
  index = sample(1:nrow(lumber), nrow(lumber), replace = T)
  betas[i,] <- coef(
    glm(customers ~ store_distance * competitor_dist,
        family = "poisson", data = lumber[index,])
  )
}
apply(betas, 2, mean) |> round(4) # Take mean of each column
[1] 4.3501 -0.3305 -1.1577 -0.9233 0.2125 0.2021
apply(betas, 2, sd) |> round(4) # Take SD of each column
[1] 0.5390 0.0723 0.5796 0.5462 0.0780 0.0741
```